Smart Vision Assist (FAVS)

Technical Project Report

Field	Current Project State
Project name	Smart Vision Assist (FAVS)
Platform	Flutter, Android-first
Primary demo device	Samsung A06 class hardware, MediaTek Helio G85
Firebase project	`smart-vision-assist-74033`
Firebase Storage bucket	`smart-vision-assist-74033.firebasestorage.app`
Object detection model	EfficientDet Lite0 / SSD-style TFLite by default
Legacy model support	YOLOv8 TFLite path remains as a fallback-compatible decoder
Languages	English and Arabic, runtime switchable
Federated learning mode	Privacy-preserving Lite FL pipeline
Report status	Source-of-truth audit updated April 2026

1. Project Overview

Smart Vision Assist (FAVS) is an Android-first accessibility application for visually impaired users. It combines real-time obstacle detection, OCR, scene description, emergency assistance, bilingual voice interaction, and walking navigation.

The system is designed around four principles:

Principle	Implementation
Accessibility first	Major workflows are spoken through `TtsHelper`, `VoiceBar`, and voice commands.
Language purity	UI, TTS, STT, object labels, and navigation prompts follow the active English/Arabic locale.
Device-first vision	Object detection runs locally through TFLite; camera frames do not need cloud processing.
Privacy-preserving learning	FL images stay on-device; Firebase receives only compact mathematical model updates.

2. Current System Architecture

```
USER / HARDWARE
  Camera, microphone, speaker, GPS
        |
        v
EDGE INTELLIGENCE LAYER
  ObstacleDetectionService
    - EfficientDet Lite0 / SSD-style output path
    - YOLO output decoder retained for compatibility
    - model metadata label extraction
    - 80-dim FL activation extraction

  NavigationScreen + NavigationService
    - bilingual destination input
    - OSRM route steps
    - pulsed safety detection every 2 seconds during guidance

  DecisionEngine
    - prioritizes safety alerts over navigation
    - suppresses low-priority obstacle speech during guidance
    - interrupts TTS for critical close-range danger

  ObjectLabelLocalizer
    - maps COCO object labels, directions, and tips into Arabic at runtime

  TtsHelper + VoiceEnabledMixin
    - synchronizes TTS/STT language with SettingsService
        |
        v
COMMUNICATION LAYER
  Firebase Auth
  Firestore user settings and FL metadata
  Firebase Storage global model download and local update upload
  Gemini API for Arabic OCR and scene description
  Nominatim/OSRM for navigation
        |
        v
CLOUD / FL AGGREGATION LAYER
  fl_server/aggregator.py
    - reads fl_model_updates/{uid}/update_*.json
    - runs weighted FedAvg over 80-dim update vectors
    - writes fl_global_model/global_weights.json
    - logs each round in fl_rounds/{round_id}
```

---

3. Object Detection Source of Truth

Current Model Path

The active default model is configured in:

```dart
// lib/core/config/app_config.dart
static String get obstacleModelAsset => const String.fromEnvironment(
  'OBSTACLE_MODEL_ASSET',
  defaultValue: 'assets/ml/efficientdet_lite0.tflite',
);
```

This means the report must describe the production path as **EfficientDet Lite0 / SSD-style TFLite**, not as a YOLO-only implementation. The code still includes YOLO decoding support so older assets can be tested, but YOLO is no longer the primary demo model.

Detection Service Behavior

Area	Current behavior
Model loading	Model is extracted from assets into app support storage, then loaded by `tflite_flutter`.
Threading	TFLite interpreter uses one CPU thread for Helio G85 stability.
Input contract	Input size and tensor type are detected dynamically from the model.
Output contract	Service detects SSD/EfficientDet multi-output tensors or YOLO single-output tensors.
Label correctness	Labels are extracted from TFLite model metadata when available; asset labels are fallback only.
Localization	English model labels are converted at runtime by `ObjectLabelLocalizer` when Arabic is active.

Why Metadata Labels Matter

The safest way to avoid label-order bugs such as "laptop" being announced as "refrigerator" is to read labels from the model metadata. The current service does this first, and only falls back to `assets/ml/coco-labels-2014_2017.txt` if the model has no embedded label file.

---

4. Bilingual UX and Navigation

FAVS supports English and Arabic through:

Component	Role
`LocaleNotifier`	Switches `MaterialApp.locale` at runtime.
`SettingsService`	Persists selected language locally and syncs it to Firestore.
`TtsHelper`	Keeps TTS language aligned with the current settings language.
`VoiceEnabledMixin`	Selects `en-US` or `ar-SA` for speech recognition.
`ObjectLabelLocalizer`	Localizes object names, directions, alert tips, and warning sentences.

Component	Role
`NavigationScreen`	Uses bilingual voice input, confirmation phrases, status text, and route summaries.

Navigation accepts Arabic or English speech depending on the active UI locale. Route calculation remains online because geocoding and walking routes are provided through Nominatim and OSRM.

---

5. Pulsed Safety Detection During Navigation

The navigation screen runs a conservative safety detector while guidance is active.

Constraint	Implementation
Avoid camera/model overload	Detection is pulsed, not continuous.
Pulse interval	One detection attempt every 2 seconds.
Frame backlog protection	`_safetyProcessing` prevents overlapping inference calls.
Safety filter	Only danger-level detections are sent to the alert engine.
TTS priority	`DecisionEngine` lets critical safety warnings interrupt navigation speech.
Helio G85 fit	Single-thread TFLite plus 2-second pulse rate keeps CPU pressure lower than continuous camera AI.

This feature is important for the demo because it shows functional interaction between two major modules: navigation and object detection.

---

6. Federated Learning Lite Pipeline

The current FL implementation is intentionally demo-safe for mobile hardware. It does not run full neural-network backpropagation on the Samsung A06. Instead, it builds a compact local update from uncertain detections.

On-Device Collection

Step	Implementation
User consent	Controlled by "Join Federated Learning" in Settings.
Sample trigger	Frames are considered only when FL is enabled.
Confidence window	Samples are collected when confidence is between 0.25 and 0.44.
Local payload	A JPEG preview and JSON feature vector are saved locally.
Threshold	At 200 local samples, the app can build a local FL update.
Privacy boundary	Raw images and per-frame samples are not uploaded to Firebase.

Local Update

`FlSampleCollector` converts 200 uncertain samples into one 80-dimensional local update:

```
local_weights[class] = average max activation for that class over 200 samples
```

This is a "Lite FL" approach. It preserves the academic FL story: local data affects a global model through numerical client updates while keeping raw data on the device.

Firebase Upload and Aggregation

Firebase path	Data
`fl_model_updates/{uid}/update_*.json`	Local mathematical update only.
`fl_contributions/{uid}/updates/{timestamp}`	Metadata and privacy flags.
`fl_global_model/global_weights.json`	Aggregated global 80-dim weights downloaded by the app.
`fl_rounds/{round_id}`	Server-side aggregation audit log.

The Python server now reads `fl_model_updates/` first and only keeps legacy `fl_samples/` support as a fallback for older test data.

---

7. Firebase Integration

Firebase is used in four separate ways:

Service	Current use
Firebase Auth	User login/signup and current-user routing.
Firestore	Settings sync and FL contribution metadata.
Firebase Storage download	`FederatedLearningService` pulls `fl_global_model/global_weights.json`.
Firebase Storage upload	`FlSampleCollector` uploads only local update JSON files.

Android Firebase configuration is present through `android/app/google-services.json` and the Gradle Google Services plugin. The Python aggregation server is configured to the same Storage bucket declared in `google-services.json`: `smart-vision-assist-74033.firebasestorage.app`. iOS Firebase configuration is not considered part of the Android-first demo unless a separate `GoogleService-Info.plist` is added.

---

8. Performance Assessment for Samsung A06 / Helio G85

Feature	Demo readiness	Notes
EfficientDet Lite0 detection	Ready	Single-thread CPU inference is appropriate for low-end hardware.
Continuous obstacle detection screen	Ready with frame dropping	The service avoids queue buildup by processing one frame at a time.
Navigation + pulsed safety detection	Ready	One scan every 2 seconds is safer than simultaneous continuous AI and route guidance.
Full on-device training	Not recommended	Too heavy for demo hardware and may cause heat or lag.
FL Lite update generation	Ready	Averaging 200 vectors is lightweight and academically defensible as a client update.
Firebase sync	Ready	Manual sync avoids background surprises and protects demo stability.

9. Android Build Configuration

The Gradle problem report showed:

```
Dependency requires at least JVM runtime version 11.
This build uses a Java 8 JVM.
```

The project now pins Gradle to Android Studio's bundled JBR:

```
# android/gradle.properties
org.gradle.java.home=C:/Program Files/Android/Android Studio/jbr
org.gradle.jvmargs=-Xmx2048m -XX:MaxMetaspaceSize=512m -Dfile.encoding=UTF-8
org.gradle.workers.max=2
org.gradle.daemon=false
```

The app module also targets Java 17:

```
// android/app/build.gradle.kts
compileOptions {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
}

kotlinOptions {
    jvmTarget = JavaVersion.VERSION_17.toString()
}
```

This resolves the Java 8 vs Java 11/17 mismatch and reduces the chance of another Gradle JVM crash on memory-limited Windows machines.

---

10. Committee Talking Points

What makes the FL system privacy-preserving?

Images are saved locally for the user's device only. Firebase receives only a compact 80-dimensional local update plus metadata. The server never receives camera frames.

Why use a Lite FL update?

Full model training on a Helio G85 phone is risky for heat, battery, and demo smoothness. The Lite update still demonstrates the core FL idea: local observations are converted into client-side numerical model updates and aggregated centrally through FedAvg.

How are object labels kept correct?

The detection service extracts label files from TFLite metadata before using fallback asset labels. This prevents mismatches caused by different model label orders.

How does Arabic support work for detections?

The model still outputs canonical English COCO labels. The app maps those labels through `ObjectLabelLocalizer` immediately before displaying or speaking them, so inference latency is unaffected.

How does the app avoid audio conflicts?

`DecisionEngine` prioritizes safety. Critical close-range obstacles stop current TTS and speak immediately, while navigation prompts are deferred after critical alerts.

---

11. Known Scope Boundaries

Area	Current status
iOS Firebase	Not configured for the Android-first demo.

Area	Current status
Heavy on-device neural training	Intentionally avoided for stability.
YOLO-only documentation	Removed as the main architecture; YOLO remains a fallback decoder only.
Automatic Wi-Fi/charging FL guard	Not implemented with plugins; current demo uses manual sync and lightweight updates.
Java build mismatch	Fixed through Gradle JVM pinning and Java 17 toolchain configuration.

12. Final Source-of-Truth Summary

FAVS is now best described as a bilingual, Android-first accessibility app using on-device EfficientDet Lite0 / SSD-style TFLite detection, runtime label metadata extraction, Arabic/English object label localization, pulsed safety detection during navigation, and a privacy-preserving Federated Learning Lite pipeline with 200-sample local update generation and Firebase-based aggregation.